Rust + Anchor Core Cheat Sheet
A practical bridge from beginner Rust to Solana programs
Keep this beside rustlings-core and anchor-core

This guide explains the concepts you will repeatedly meet while writing Anchor programs. Read it as a map, not something to memorize. The central idea is that a Solana program is a deterministic Rust state machine operating on accounts supplied by a caller. Correctness comes from validating those accounts before changing state.

The five questions to ask for every instruction

1. What state may change?
2. Who is authorized to request the change?
3. Does every account have the expected address, owner, type, and permissions?
4. Which invariant must remain true after the change?
5. Can arithmetic, allocation, a CPI, or compute usage fail?

Rust values, variables, and mutability

"let" binds a name to a value. Values are immutable unless the binding uses "mut". Immutability makes state changes visible and easier to audit.

```
let balance = 10_u64;
let mut counter = 0_u64;
counter += 1;
```

Anchor mapping: an account must be marked mutable before its data or lamports can change. Rust "mut" controls a local binding; Anchor's "mut" account constraint tells the runtime and transaction that the account will be written. They are related ideas but are not interchangeable.

Scalar types and arithmetic

Use explicit integer types for stored protocol state. Unsigned integers (u8, u16, u32, u64, u128) cannot represent negative values. Their size controls both range and account space. Never rely on release-mode wrapping for financial math.

```
let next = balance.checked_add(deposit)
    .ok_or(ProtocolError::Overflow)?;
let remaining = balance.checked_sub(withdrawal)
    .ok_or(ProtocolError::InsufficientFunds)?;
```

For ratios and prices, define a fixed-point scale and rounding direction. Avoid floating point in protocol state. Rounding is economic policy, not a cosmetic implementation detail.

Ownership: who must clean up a value?

Every Rust value has one owner. Moving a non-Copy value transfers ownership. When the owner leaves scope, Rust drops the value. Small scalar types such as u64 usually implement Copy, so assignment copies their bits.

```
let name = String::from("vault");
let moved = name;          // name can no longer be used
let amount = 10_u64;
let copied = amount;       // both remain usable
```

Rust ownership is different from Solana account ownership. Rust ownership is a compile-time memory rule. A Solana account's owner field names the program allowed to modify its data. Always check both concepts in their proper layer.

Borrowing and references

"&T" is a shared, read-only borrow. "&mut T" is an exclusive mutable borrow. Rust permits many readers or one writer at a time. A borrow does not take ownership and normally avoids a copy.

```
fn balance(vault: &Vault) -> u64 { vault.balance }
fn freeze(vault: &mut Vault) { vault.frozen = true; }
```

Anchor's Context contains references to validated accounts. Instruction handlers usually borrow account data, update it, then return. Exclusive borrows help prevent two pieces of Rust code from mutating the same value at once, but they do not replace on-chain account validation.

Strings and slices

String owns growable UTF-8 bytes. &str borrows UTF-8 text. [T] is a sequence of items, and &[T] borrows such a sequence. Slices are views, so they avoid new allocations.

```
fn prefix(text: &str, delimiter: char) -> &str {
    &text[..text.find(delimiter).unwrap_or(text.len())]
}
```

String indexing uses byte offsets and may panic if an offset splits a UTF-8 character. For protocol inputs, limit maximum lengths before doing expensive work. Anchor/Borsh stores a String as a 4-byte length followed by UTF-8 bytes; allocate for the maximum encoded length, not today's value.

Structs, impl blocks, and state

A struct groups fields into one type. An impl block defines associated functions and methods. A method with &mut self changes the value.

```
struct Vault { authority: Pubkey, balance: u64, frozen: bool }
impl Vault {
    fn can_withdraw(&self, signer: Pubkey) -> bool {
        !self.frozen && self.authority == signer
    }
}
```

Anchor mapping: a struct marked as an account describes serialized on-chain state. Keep stored state minimal, version it deliberately, and document each field's invariant.

Enums and exhaustive match

An enum represents one of several valid alternatives. Variants may carry data. "match" forces all cases to be handled, which is ideal for state machines.

```
enum Command { Deposit(u64), Withdraw(u64), Freeze }
match command {
    Command::Deposit(n) => deposit(n),
    Command::Withdraw(n) => withdraw(n),
    Command::Freeze => freeze(),
}
```

Protocol states should make invalid transitions explicit. A frozen vault should

not silently behave like an active vault. Prefer a clear enum or explicit flag plus validation over magic numbers.

Option, Result, and the question-mark operator

Option<T> means a value may be present or absent. Result<T, E> means an operation either succeeds with T or fails with E. The ? operator returns an error early while preserving the success value.

```
fn debit(balance: u64, amount: u64) -> Result<u64, Error> {
    balance.checked_sub(amount).ok_or(Error::InsufficientFunds)
}
```

Do not use unwrap or expect on untrusted instruction or account data. A panic produces a poor failure boundary. Return a specific program error that clients and tests can understand.

Traits and generics

A trait defines behavior that multiple types can implement. A generic function works with types satisfying stated trait bounds.

```
trait Validate { fn validate(&self) -> Result<(), Error>; }
fn checked<T: Validate>(value: &T) -> Result<(), Error> {
    value.validate()
}
```

Anchor uses traits heavily for serialization, deserialization, account loading, and instruction construction. You need to read trait bounds confidently; you do not need advanced trait wizardry to begin writing programs.

Collections and bounded work

Vec<T> is a growable sequence. Arrays [T; N] have a compile-time length. HashMap is useful off-chain but deterministic ordered representations are often easier to reason about in protocol logic. Every on-chain collection needs an explicit maximum size or a separate account-growth strategy.

Unbounded input creates three risks: insufficient account space, excessive compute, and denial of service. Validate length before loops, allocation, hashing, or CPIs.

Iterators and closures

Iterators process sequences through methods such as map, filter, find, any, and fold. They are usually zero-cost abstractions, but collect allocates a new collection.

```
let total = positions.iter()
    .try_fold(0_u64, |sum, p| sum.checked_add(p.amount))
    .ok_or(Error::Overflow)?;
```

On-chain, a concise iterator is not automatically cheaper than a loop. Measure compute units for hot paths and avoid repeated scans when state can maintain a safe aggregate.

Lifetimes

A lifetime describes how long a reference remains valid. It does not extend the life of data; it records a relationship the compiler must enforce.

```
fn choose<'a>(left: &'a str, right: &'a str, first: bool) -> &'a str {
    if first { left } else { right }
}
```

In Anchor, ’info connects account references inside Context and account wrapper types. Read it as "these references are valid for the instruction’s account context." Start by following compiler guidance; explicit lifetime complexity is usually needed only in helpers involving several account references.

Modules, visibility, and crates

A crate is a compilation unit. Modules organize code. Items are private unless made pub. Keep instruction handlers small and place pure state-transition logic in testable modules. Expose only the API other modules truly need.

Common program layout: instructions, state, errors, constants, events, and utility validation. Structure is useful only when it makes security boundaries easier to see.

Solana account mental model

An account has an address, lamports, an owner program, executable status, and a byte array. Programs are stateless code; persistent application state lives in accounts. A transaction names all accounts an instruction may read or write.

Address

Identity of the account. Validate expected fixed addresses or derivations.

Owner

Program allowed to modify account data. This is not the user authority.

Authority

A protocol-defined key allowed to perform an action, often required to sign.

Signer

Evidence that the transaction was authorized by a private key.

Writable

Permission declared by the transaction for state mutation.

These properties answer different questions. Checking only one is rarely enough.

Anchor Context and constraints

Context groups the accounts for an instruction. The Accounts-derived struct and its constraints form the first validation wall. Useful constraints include signer, mut, owner, address, has_one, seeds, bump, constraint, init, close, and token-specific checks.

Constraints reduce boilerplate but do not invent your business invariants. You must still validate amounts, state transitions, timing, limits, relationships, and any assumptions about accounts used by a CPI.

PDAs

A Program Derived Address is deterministically calculated from seeds, a program ID, and a bump. It has no private key. The deriving program can sign for it during a CPI by supplying the correct seeds.

Good seeds create one clear namespace and bind state to the correct domain, for example ["position", market, user]. Validate seeds and bump on every instruction that trusts the PDA. Seed design is part of the protocol’s authorization model. Never treat knowing a PDA address as proof of user authorization.

Cross-program invocations

A CPI asks another program to execute an instruction. Treat the target program and every forwarded account as a trust boundary. Verify the program ID, provide only required signer privileges, validate accounts before and after when needed, and understand which state the callee may change.

A transaction is atomic: if a CPI fails and the error is propagated, the entire transaction rolls back. Atomicity does not make a logically incorrect CPI safe.

Serialization and account space

Anchor account data normally starts with an 8-byte discriminator. Fixed-size integers occupy their byte width. A bool is 1 byte. Pubkey is 32 bytes. String and Vec include a 4-byte length prefix plus maximum contents. Option adds a tag plus its possible value. Enums require a variant tag plus enough room for their largest variant.

```
space = 8                  // discriminator
    + 32                   // authority Pubkey
    + 8                    // balance u64
    + 1                    // frozen bool
    + 4 + MAX_NAME_BYTES;  // String
```

Do not calculate dynamic space from only the initial input if future updates may grow the value. Reallocation and rent funding need explicit design.

Transactions, instructions, and atomicity

An instruction contains a program ID, an account metadata list, and instruction data. A transaction contains one or more instructions plus signatures and a recent blockhash. All instructions succeed or the transaction fails atomically.

Transactions can be retried by clients. Design actions to reject duplicate or stale operations when repetition would be harmful. Use sequence numbers, receipts, unique PDAs, or explicit state transitions where appropriate.

Compute units and performance

Solana execution is metered. Expensive patterns include unbounded loops, repeated serialization, large allocations, excessive logging, repeated PDA derivation, and unnecessary CPIs. Optimize after correctness, then measure.

Fast Rust habits that also improve clarity: borrow instead of clone, return slices where possible, validate cheap conditions before expensive work, avoid collect when streaming is enough, and store safe aggregates rather than rescanning large collections.

Testing strategy

1. Test the happy path and exact state changes.
2. Test each authorization failure: wrong signer, missing signer, wrong authority.
3. Test account substitution: wrong owner, wrong PDA seeds, wrong program ID.
4. Test boundaries: zero, one, maximum, maximum plus one, overflow, underflow.
5. Test invalid state transitions and repeat the same instruction.
6. Test CPI failure and confirm state remains consistent.
7. Measure compute on realistic maximum-size inputs.

Prefer pure Rust unit tests for state math and validation. Add program-level tests for account constraints, serialization, PDAs, CPIs, and transaction behavior.

Security checklist before every state change

Authorization

Did the correct authority sign, and is that authority stored or derived correctly?

Identity

Are fixed addresses, PDA seeds, bump, and account relationships verified?

Ownership

Is each data account owned by the expected program?

Type

Is data length and discriminator correct before deserialization?

Mutability

Are only intended accounts writable, and can aliases/duplicates cause trouble?

Arithmetic

Are checked operations, units, precision, and rounding direction explicit?

State

Is this transition legal from the current state, including repeated calls?

CPI

Is the target program trusted and are forwarded privileges minimal?

Resources

Are input length, account growth, compute, and fees bounded?

Closing

Does closing send lamports to the intended recipient and prevent account revival?

Common confusions

Rust

The variable responsible for a value’s memory.

Account

The program ID allowed to modify an account’s data.

Authority

A key your protocol recognizes as permitted to perform an action.

Signer

A transaction participant that proved control of a private key.

mut

Allows mutation through a binding or reference.

mut

Declares an account writable for the transaction.

PDA

A program signs through runtime-verified seeds during a CPI; no secret key exists.

Recommended Rustlings Core expansion

The existing six exercises cover mutability, ownership, borrowing, state models, Result, and slices. Add these exercises next:

ex07

Collections: Vec, arrays, maps, maximum lengths, and avoiding needless clones.

ex08

Traits and generics: reusable validation with simple trait bounds.

ex09

Iterators: find, any, try_fold, and allocation awareness.

ex10

Lifetimes: returned borrows and Anchor-style Context relationships.

ex11

Checked arithmetic: overflow, underflow, fixed-point scale, and rounding.

ex12

Serialization and space: manual byte layout and maximum account size.

ex13

Modules and tests: library boundaries, unit tests, and adversarial cases.

ex14

Security invariants: signer, authority, owner, PDA, duplicates, and legal transitions.

Learning loop

1.

Read the test before the implementation.

2.

Predict ownership, state, and expected error on paper.

3. Write the smallest correct implementation.

4. Explain why each validation exists and what attack it blocks.

5. Add one failure test the exercise did not provide.

6. Translate the Rust idea into its Anchor/account equivalent.

You are ready to build educational protocols when you can implement a vault or escrow locally, explain every account constraint, test all failure paths, and state the protocol invariants in plain language. Real funds require substantially more review, adversarial testing, and independent security assessment.